

CS-405: Deep Learning

Week 4 - Deep Convolutional Neural Networks Architectures

Dr. Sajjad Hussain

February 10th, 2026

Based on: AlexNet (Krizhevsky 2012), VGG (Simonyan & Zisserman 2014),
ResNet (He 2015), MobileNet (Howard 2017, Sandler 2018)

Outline

- 1 AlexNet (2012) – The Breakthrough
- 2 VGGNet (2014) – Simplicity and Depth
- 3 1×1 Convolution – The "Channel Mixer"
- 4 ResNet (2015) – Learning Residuals
- 5 Inception – Multi-Scale Processing
- 6 Depthwise Separable Convolution
- 7 MobileNet V1
- 8 MobileNet V2 – Inverted Residuals
- 9 Design Evolution and Comparison

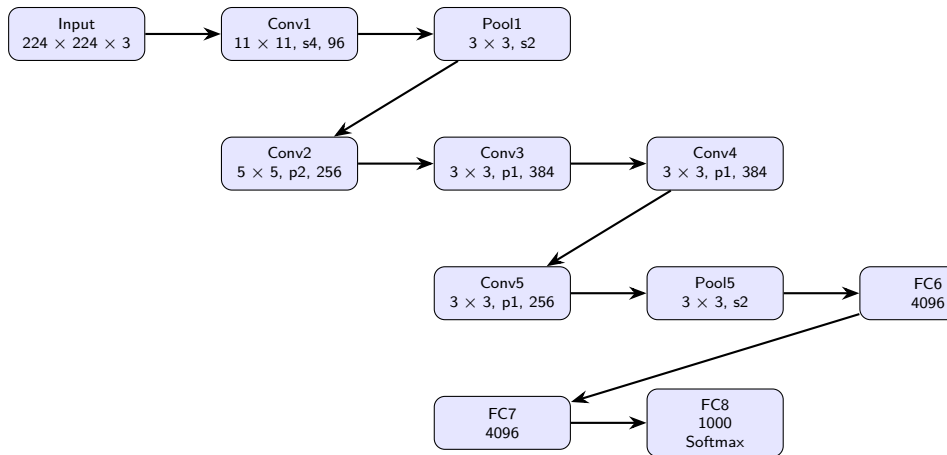
Context: Why AlexNet?

- Before 2012, CNNs were considered slow and shallow.
- AlexNet won ImageNet 2012 (top-5 error 15.3% vs second place 26.2%).

Context: Why AlexNet?

- Before 2012, CNNs were considered slow and shallow.
- AlexNet won ImageNet 2012 (top-5 error 15.3% vs second place 26.2%).
- Key innovations:
 - **ReLU** non-linearity.
 - **GPU** implementation.
 - **Dropout**.
 - **Data augmentation**.

AlexNet Architecture



- LRN (Local Response Normalization) used originally, now deprecated.
- 60 million parameters; 2 GPUs split feature maps.

VGGNet: Replacing Large Kernels

Motivation: Replace large filters (11×11 , 5×5) with stacked 3×3 convolutions.

VGGNet: Replacing Large Kernels

Motivation: Replace large filters (11×11 , 5×5) with stacked 3×3 convolutions.

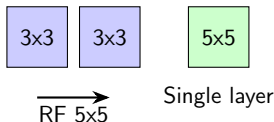
Why two 3×3 layers?

- Two 3×3 conv (stride 1) have effective receptive field 5×5 .
- Three 3×3 conv $\rightarrow 7 \times 7$ receptive field.

Example (Parameter advantage)

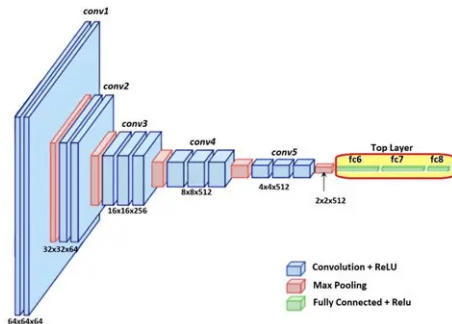
3×3 : $3^2 C^2 = 9C^2$ parameters (two layers) vs 5×5 : $25C^2$ parameters.

Reduction: $\frac{18}{25} \approx 28\%$ less.



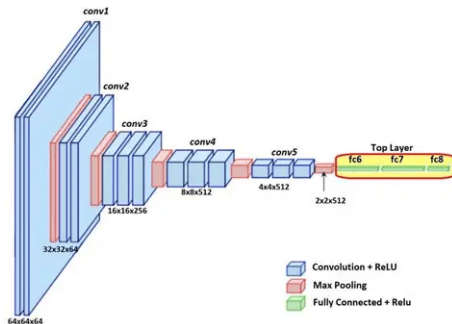
VGG-16 Architecture

- **Config:** 13 conv + 3 FC.
- Fixed: 3×3 conv, stride 1, pad 1; 2×2 maxpool, stride 2.
- Channel doubling after each pool: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$.



VGG-16 Architecture

- **Config:** 13 conv + 3 FC.
- Fixed: 3×3 conv, stride 1, pad 1; 2×2 maxpool, stride 2.
- Channel doubling after each pool: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$.

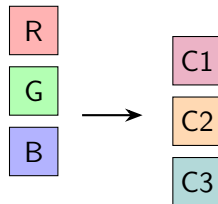


138M parameters; very expensive in FC layers.

1×1 Convolution: What Does It Do?

Think of it as "Channel Blending"

- Imagine each pixel has multiple values (channels) like R,G,B
- A 1×1 convolution looks at **one pixel at a time**
- It takes all channels of that pixel and mixes them together
- Produces new values for that same pixel

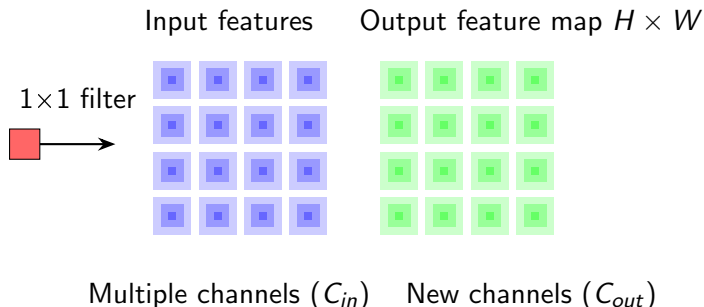


Simple Analogy

Like adjusting the color mix on a TV:

You take the same pixel but change how its color channels combine

1×1 Convolution: How It Works Visually



- **Key insight:** The filter never looks at neighboring pixels
- Each output pixel depends only on the input pixel at the **same position**
- Think of it as "per-pixel channel remixing"

1×1 Convolution: Mathematical View

Operation

Input: $H \times W \times C_{in}$; Filter: $1 \times 1 \times C_{in} \times C_{out}$.

- No spatial aggregation; purely channel-wise linear combination + non-linearity.
- Equivalent to applying a fully connected layer to each pixel (shared weights across space).

1×1 Convolution: Mathematical View

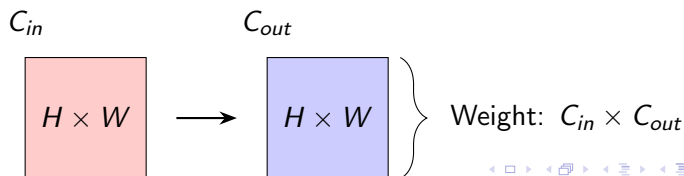
Operation

Input: $H \times W \times C_{in}$; Filter: $1 \times 1 \times C_{in} \times C_{out}$.

- No spatial aggregation; purely channel-wise linear combination + non-linearity.
- Equivalent to applying a fully connected layer to each pixel (shared weights across space).

Dimensionality reduction / expansion

- $C_{out} < C_{in}$: bottleneck (reduce channels).
- $C_{out} > C_{in}$: expansion (increase channels).



Computational Cost: 1×1 vs Larger Kernels

Standard Conv FLOPs:

$$\text{Cost} = H \times W \times C_{in} \times C_{out} \times K^2$$

For $K = 1$:

$$\text{Cost}_{1 \times 1} = H \times W \times C_{in} \times C_{out}$$

Computational Cost: 1×1 vs Larger Kernels

Standard Conv FLOPs:

$$\text{Cost} = H \times W \times C_{in} \times C_{out} \times K^2$$

For $K = 1$:

$$\text{Cost}_{1 \times 1} = H \times W \times C_{in} \times C_{out}$$

- Negligible spatial cost, only channel mixing.
- Critical for efficient blocks (Inception, MobileNet, ResNet bottleneck).

1×1

3×3 conv: needs $9 \times$ more computation

The Degradation Problem

- Intuition: deeper $\rightarrow$ better. But plain nets show **higher training error** when depth increases.

The Degradation Problem

- Intuition: deeper $\rightarrow$ better. But plain nets show **higher training error** when depth increases.
- Not caused by vanishing gradients (thanks to BN/ReLU).
- **Hypothesis:** Hard to fit identity mapping via stacked nonlinear layers.

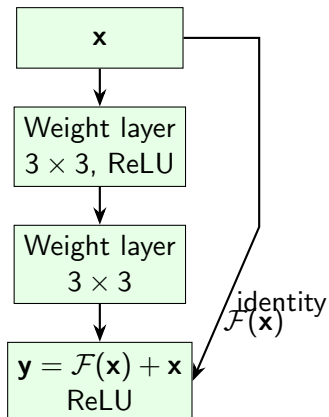
Solution: Residual learning.

Residual Block

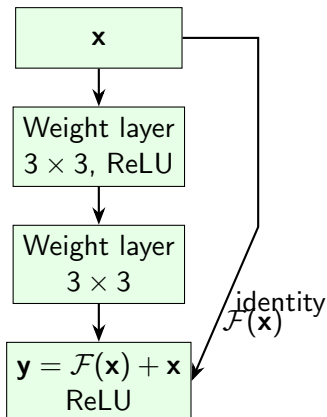
$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

- $\mathcal{F}$ = stacked conv layers (usually 2 or 3 layers).
- $\mathbf{x}$ = identity skip connection.

Residual Block Diagram



Residual Block Diagram



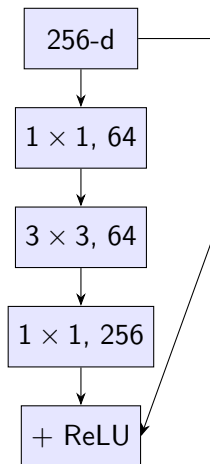
Gradient Propagation

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(1 + \frac{\partial \mathcal{F}}{\partial \mathbf{x}} \right)$$

Gradient flows directly through skip connection; never vanishes.

Bottleneck Design in ResNet

- For deeper nets (ResNet-50/101/152), use **bottleneck** block (i.e. 1×1 convolution) to reduce computation.



Bottleneck Design: Simple Explanation

The Core Idea

Instead of applying expensive 3×3 convolutions on many channels:

- 1 **First compress** the channels ($256 \rightarrow 64$) using a cheap 1×1 conv
- 2 **Then process** with 3×3 conv on the compressed representation
- 3 **Finally expand** back ($64 \rightarrow 256$) using another cheap 1×1 conv

Like summarizing a document before detailed analysis, then expanding the findings

Sample Computations: Plain vs Bottleneck

Plain Block (Direct):

$$256 \xrightarrow{3 \times 3} 256$$
$$\text{Cost} = \underbrace{256}_{C_{in}} \times \underbrace{256}_{C_{out}} \times \underbrace{9}_{K^2} = \mathbf{589,824} \text{ (per pixel)}$$

Bottleneck (Efficient):

$$256 \xrightarrow{1 \times 1} 64 \xrightarrow{3 \times 3} 64 \xrightarrow{1 \times 1} 256$$
$$\begin{aligned} \text{Cost} &= \underbrace{256 \times 64 \times 1}_{\text{squeeze}} + \underbrace{64 \times 64 \times 9}_{\text{process}} + \underbrace{64 \times 256 \times 1}_{\text{expand}} \\ &= 16,384 + 36,864 + 16,384 = \mathbf{69,632} \text{ (per pixel)} \\ \frac{589,824}{69,632} &\approx 8.5 \times \text{fewer operations!} \end{aligned}$$

- The 3×3 convolution now works on only **64 channels** (not 256) - that's the key saving!
- 1×1 convolutions are cheap: no sliding, just channel mixing

Inception Motivation: Multi-scale

- Objects appear at different scales within same image.
- Instead of choosing kernel size, let the network learn.
- **Naive Inception:** parallel 1×1 , 3×3 , 5×5 conv, 3×3 maxpool, concatenate.

Inception Motivation: Multi-scale

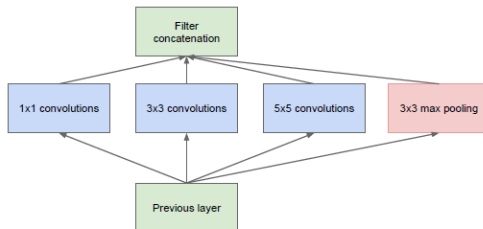
- Objects appear at different scales within same image.
- Instead of choosing kernel size, let the network learn.
- **Naive Inception:** parallel 1×1 , 3×3 , 5×5 conv, 3×3 maxpool, concatenate.

Computational explosion

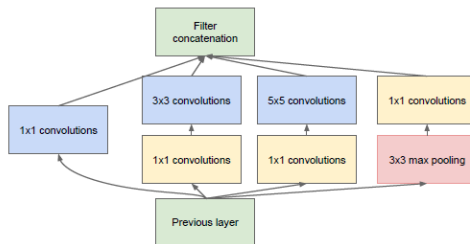
5×5 conv on high-channel input is expensive.

Solution: Add 1×1 bottleneck before expensive convs.

Inception Module with Dimension Reduction



(a) Inception module, naïve version



(b) Inception module with dimensionality reduction

GoogLeNet (Google Inception)

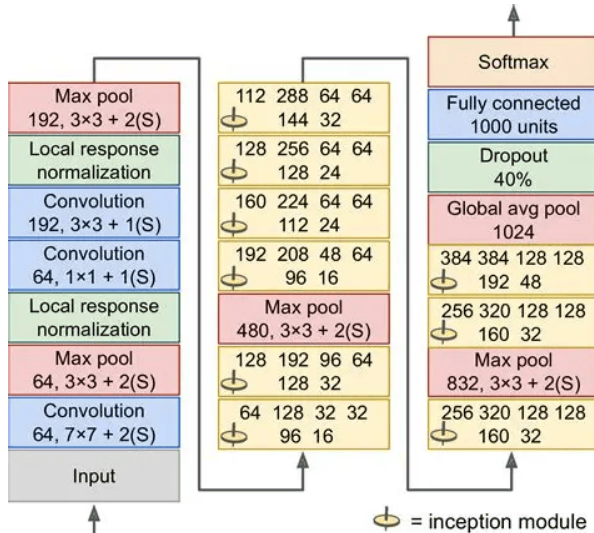
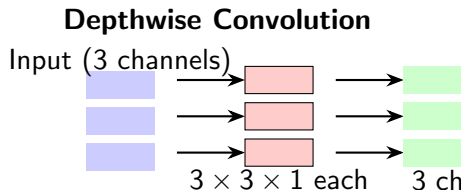
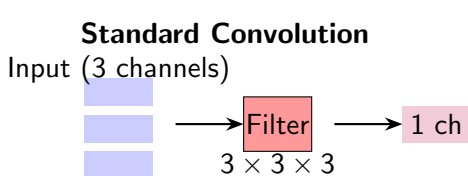


Figure: Inception Network

Depthwise Convolution: What Does It Do?

Think of it as "Per-Channel Processing"

- Regular convolution mixes channels AND spaces simultaneously
- Depthwise convolution treats each channel **separately**
- Like having a separate 2D filter for each channel



Depthwise Separable: Two-Step Process

Step 1: Depthwise Convolution (Spatial Only)

- Apply one 3×3 filter **per input channel**
- No mixing between channels - each channel stays separate
- Output has same number of channels as input

Depthwise Separable: Two-Step Process

Step 1: Depthwise Convolution (Spatial Only)

- Apply one 3×3 filter **per input channel**
- No mixing between channels - each channel stays separate
- Output has same number of channels as input

Step 2: Pointwise (1×1) Convolution (Channel Mixing)

- Now use 1×1 convolution to mix channels
- Combine the separate channel outputs into new representations
- Creates desired number of output channels

Simple Example: Let's Count the Operations

Setup

Input: 7×7 image with 3 channels ($C_{in} = 3$)

Output: Want 4 channels ($C_{out} = 4$) using 3×3 filters

Standard Convolution:

$$\begin{aligned}\text{Cost} &= H \times W \times C_{in} \times C_{out} \times K^2 \\ &= 7 \times 7 \times 3 \times 4 \times 9 \\ &= \mathbf{5,292} \text{ operations}\end{aligned}$$

One step: mix spaces AND channels

Depthwise Separable:

$$\begin{aligned}\text{Depthwise:} &= H \times W \times C_{in} \times K^2 \\ &= 7 \times 7 \times 3 \times 9 = 1,323\end{aligned}$$

$$\begin{aligned}\text{Pointwise:} &= H \times W \times C_{in} \times C_{out} \\ &= 7 \times 7 \times 3 \times 4 = 588\end{aligned}$$

$$\mathbf{Total} = 1,323 + 588 = \mathbf{1,911}$$

Reduction: $\frac{5,292}{1,911} \approx 2.8\times$ fewer operations!

Why Such Big Savings?

- **Standard conv:** Does two things at once (spatial + channel mixing)
 - expensive!
- **Depthwise sep:** Splits into two cheaper steps

General Formula

$$\frac{\text{Cost}_{sep}}{\text{Cost}_{standard}} = \frac{1}{C_{out}} + \frac{1}{K^2}$$

Example: $K = 3$, $C_{out} = 4$ (our example)

$$\frac{1}{4} + \frac{1}{9} = 0.25 + 0.111 = 0.361 \rightarrow 2.8 \times \text{reduction}$$

With many channels: $K = 3$, $C_{out} = 512$

$$\frac{1}{512} + \frac{1}{9} \approx 0.002 + 0.111 = 0.113 \rightarrow 8.8 \times \text{reduction!}$$

Summary: Depthwise Separable Convolution

Step 1: Depthwise

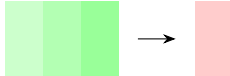
3×3 each



C_{in} filters

Step 2: Pointwise

1×1 each



C_{out} filters

Total Savings

$$\frac{1}{C_{out}} + \frac{1}{K^2}$$

With big C_{out} :

$$\approx \frac{1}{K^2}$$

Key Takeaway

Depthwise separable convolution = **spatial processing** (depthwise) + **channel mixing** (pointwise)

≈ 8 – 9× cheaper for deep networks!

MobileNet V1: Depthwise Separable Stack

- First architecture built entirely on depthwise separable convolutions.
- Width multiplier α : thin channels; resolution multiplier ρ : reduce input size.

$224 \times 224 \times 3$

Conv 3×3 , s2, 32

Depthwise 3×3 , s1

Pointwise 1×1 , 64

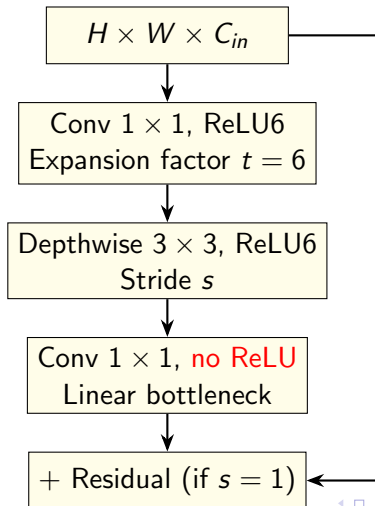
... repeat ...

AvgPool 7×7

FC 1000

MobileNet V2: Key Ideas

- **Inverted residual:** Thin $\rightarrow$ expand $\rightarrow$ depthwise $\rightarrow$ project.
- **Linear bottleneck:** No ReLU after projection (preserve information).



Practical Deployment: Why MobileNet Matters

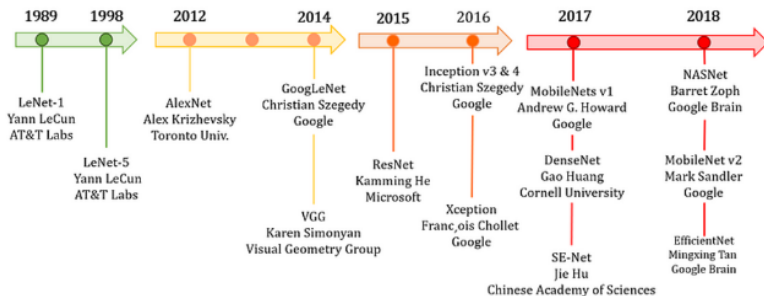
- **Edge devices:** mobile phones, drones, wearables.
- **Limits:** power, memory, compute.
- Depthwise separable: $8\text{-}9\times$ less FLOPs than standard conv.
- MobileNetV2: linear bottleneck preserves representation; inverted residual improves gradient flow.

Key takeaway

Depthwise + pointwise = standard conv replacement; Inverted + linear = efficient mobile design.

Timeline of Architectural Innovations

CNN Timeline (1989 ~ 2018)



Computational Complexity & Parameter Comparison

Model	Top-1 Acc	Params (M)	FLOPs (B)
AlexNet	57.1%	62.3	0.7
VGG-16	71.5%	138.3	15.5
ResNet-50	75.2%	25.6	3.9
Inception-v3	78.8%	23.8	5.7
MobileNetV1	70.6%	4.2	0.57
MobileNetV2	72.0%	3.4	0.30

Computational Complexity & Parameter Comparison

Model	Top-1 Acc	Params (M)	FLOPs (B)
AlexNet	57.1%	62.3	0.7
VGG-16	71.5%	138.3	15.5
ResNet-50	75.2%	25.6	3.9
Inception-v3	78.8%	23.8	5.7
MobileNetV1	70.6%	4.2	0.57
MobileNetV2	72.0%	3.4	0.30

- VGG: huge memory, slow.
- ResNet: efficient bottleneck, skip connections.
- MobileNet: depthwise separable → mobile deployment.

Summary – Advanced CNN Design Principles

- ① **Small kernels:** stack 3×3 (VGG).
- ② **1×1 conv:** channel mixing, dim reduction, cheap.
- ③ **Residual connections:** solve degradation, gradient highway.
- ④ **Bottleneck:** 1×1 compress, 3×3 , 1×1 expand (ResNet, Inception).
- ⑤ **Multi-scale:** parallel filters of different sizes (Inception).
- ⑥ **Depthwise separable:** spatial then channel (MobileNet).
- ⑦ **Inverted residual + linear bottleneck:** expand then compress, no ReLU on output (MobileNetV2).

Key Takeaways

- Deeper doesn't automatically work – need skip connections.
- Computational efficiency = smart use of 1×1 conv.
- Understanding FLOPs guides architecture design.
- Mobile architectures are optimized for real-world constraints.

References I

-  A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NIPS, 2012.
-  K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR, 2015.
-  K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.
-  C. Szegedy et al., "Going deeper with convolutions," CVPR, 2015.
-  A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv, 2017.
-  M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.